

Summary:

Project Description

E-Commerce Order Analytics System

A Python + SQL data pipeline that simulates how a company turns messy, real-world order data into business insights.

What it does:

- Generates realistic e-commerce data (customers, products, orders, order items) with intentional data-quality issues — missing IDs, bad date formats, invalid emails, negative quantities, broken references
- Cleans and validates that data with dedicated Python functions, logging every issue found and fixed
- Loads the cleaned data into a SQLite database
- Runs 16 SQL queries against it — from basic aggregations to advanced window functions, CTEs, cohort analysis, and customer segmentation
- Provides a command-line tool to generate on-demand daily/weekly/monthly business reports
- Includes automated tests proving the system handles edge cases (orphaned records, invalid discounts, zero quantities, future dates) correctly

Tech used: Python (pandas, Faker), SQLite, SQL (window functions, CTEs, subqueries), stdlib-only CLI tooling.

Purpose: demonstrates an end-to-end data analyst/engineer workflow — raw data → cleaning → database → analysis → reporting — the kind of task given to interns to test practical SQL and Python skills together.

Outputs:

```
C:\Users\radha\Downloads\e-commerce_analytics (1)\e-commerce_analytics>python run_all.py

=====
Generating raw data
=====
Generating e-commerce sample data (with intentional data-quality issues)...
wrote 600 rows -> data/customers.csv
wrote 250 rows -> data/products.csv
wrote 3000 rows -> data/orders.csv
wrote 7461 rows -> data/order_items.csv

Done. Raw data is in ./data/
Known issues injected on purpose:
- customers.csv : ~2% invalid emails
- orders.csv : ~5% missing customer_id, ~8% DD-MM-YYYY dates
- order_items : ~3% negative quantity, 15 rows with orphan order_id, a few discount_percent > 100
- products.csv : ~15% product_name with extra spaces / inconsistent case
=====
```

```
=====
Cleaning data
=====
Loading raw CSVs...

Running cleaning + validation functions...

[validate_emails] Found 7 customers with an invalid email address (1.2% of all customers).
[check_referential_integrity] Found 15 order_items rows referencing an order_id that does not exist in orders.csv. Affected order_ids: [4162, 4484, 4835, 5320, 5592, 5684, 6798, 7124, 7288, 7307, 7687, 7704, 7823, 7845]
[extra] 235 order_items rows have negative quantity (interpreted as returns, kept in data but flagged).
[extra] 0 order_items rows have quantity == 0 (these contribute zero revenue; flagged for review).
[extra] 35 order_items rows have an out-of-range discount_percent (<0 or >100). These will be CLIPPED to [0, 100] in the cleaned file.
[clean_orders] Reformatted 239 order_date values that were not in YYYY-MM-DD HH:MM:SS format (mostly DD-MM-YYYY).
[clean_orders] Found 148 orders with a missing customer_id (4.9% of all orders). These rows are KEPT (an order can still be analyzed) but customer_id is NULL rather than guessed.
[clean_products] Normalized 43 product_name values (trimmed whitespace, collapsed internal spaces, applied Title Case).
[main] Removed 15 orphan order_items rows from the cleaned dataset (they cannot be joined to any real order).

Cleaned files written to data\cleaned/
Data quality report written to reports\data_quality_report.txt
```

```
=====
Loading into SQLite
=====

customers      600 rows loaded
products       250 rows loaded
orders         3000 rows loaded
order_items    7466 rows loaded

SQLite database ready at ./ecommerce.db

=====
Running SQL analysis
=====
Wrote 17 query results to reports/sql_query_results.txt
```

```
=====
Running edge-case tests
=====
Running edge-case tests...

test_orphan_order_id:
PASS: orphan_order_id is detected and removed from cleaned data; it is reported in the data-quality report.

test_discount_over_100:
PASS: discount_percent > 100 is clipped to 100 and flagged; revenue can never go negative as a result.

test_zero_quantity:
PASS: quantity == 0 contributes $0 revenue (no crash / no divide-by-zero) and is flagged for manual review as an unusual-but-valid row.

test_future_order_date:
PASS: future_order_date is detected via a simple date-vs-now comparison and would be flagged for review rather than silently included in trend calculation s.

=====
All 4 edge-case tests passed.

All steps completed successfully.
Try the interactive report tool next: python 05_report_cli.py
```

```
C:\Users\radha\Downloads\ecommerce_analytics (1)\ecommerce_analytics>python 05_report_cli.py --type monthly --start 2024-06-01 --end 2024-06-30
=====
E-COMMERCE SUMMARY REPORT (MONTHLY)
Period: 2024-06-01 to 2024-06-30
=====

Total Orders      : 80
Total Revenue     : $179,403.89
Unique Customers  : 74

Top 3 Products by Revenue:
1. Indicate Footwear      $9,017.97
2. To Accessorie          $5,343.74
3. But Footwear           $5,155.31

Comparison with Previous Period:
Orders      :      80.00 vs      86.00  (~7.0%)
Revenue     : 179,403.89 vs 220,517.55 (~18.6%)
Unique Customers :    74.00 vs    76.00  (~2.6%)
=====
```